

Comparing Dissemination Strategies for Delta CRDTs

DIMITRIOS BAKALIS

PAUL MICHAEL KILIAN

bakalis | pmkilian@kth.se

October 24, 2025

Abstract

Weaker consistency level algorithms like Conflict-Free Replicated Data Types (CRDTs) have been developed to enable better availability in replicated Peer-to-Peer systems. The deployment of the right algorithms is key to creating more resilient, fault-tolerant, and efficient applications. In this project, we investigate the performance of different dissemination strategies (proactive, reactive, hybrid) on Delta State-based CRDTs (δ -CRDTs) in particular, across various overlay networks. For this purpose, we implemented a containerized testbed that allows for controlled experiments. Our results indicate that proactive strategies converge the fastest, but introduce higher network load and computation times in dense networks, compared to reactive strategies, which have slow convergence but high efficiency. Hybrid methods provide a compromise. This highlights the trade-offs for system operation, providing guidance for optimizations based on the results.

Keywords: Distributed computing, Data dissemination, Eventual consistency, Conflict-free replicated data types

Artifact Availability: https://github.com/kilianpl4/delta_crdt_dissemination

Contents

1	Introduction	2
1.1	Research Gap	2
1.2	Related Work	3
1.2.1	Delta state-based CRDTs	3
1.2.2	Comparative Studies of Gossiping Protocols	3
2	Method	3
2.1	Testbed Architecture	4
2.2	Overlay Networks	5
2.3	Dissemination Strategies	5
2.4	Metrics Collection	6
3	Results	6
3.1	Network Load	7
3.2	Computation Times	8
3.3	Convergence Speed	9
4	Discussion	9
5	Conclusion and Outlook	10
A	List of Acronyms and Abbreviations	12

1 Introduction

In Peer-to-Peer (P2P) systems, replication of data objects is an essential technique to achieve resilience and fault-tolerance in the absence of centralized control. Maintaining a strongly consistent view on the data in such environments, where high availability needs to be ensured and large network delays or network partitions are common, turns out to be impossible due to the constraints outlined in the CAP theorem [1]. To maintain a high level of responsiveness, such P2P systems therefore have to employ a weaker consistency model. A widely adopted one is the *(Strong) eventual consistency* model [2], which guarantees that all replicas will *eventually* converge to the same state. This means that compared to traditional consensus-based approaches like Raft [3], data might not immediately be visible across replicas.

While eventual consistency ensures that replicas will synchronize over time, it does not describe a mechanism for how concurrent updates are handled. This is the purpose of *Conflict-Free Replicated Data Types (CRDTs)*, which are data structures with specific mathematical properties designed to achieve eventual consistency through built-in algorithms [4]. Nowadays, common CRDT data types are present, for instance, in distributed NoSQL databases like Redis, RiakDB, and Azure CosmosDB. There are two main approaches to CRDTs. When using state-based CRDTs, the replicas exchange their complete state at every synchronization step and a merge function is used to integrate a remote state into a local state. In operation-based CRDTs, individual updates are propagated through reliable broadcast, and have to be delivered based on a causal order mechanism [5].

How to effectively make use of CRDTs for replicated data objects in P2P systems is still an open research topic. For the state-based approach, the size of the state increases monotonically and includes a lot of redundancy for every synchronization, whereas the operation-based approach might cause a lot of unnecessary load when many updates concern the same part of the state, constantly overriding each other [5]. Due to these constraints, new approaches in CRDT research focus on determining a representation of recent operations' effect [6, 7], and thus combine state-based and operation-based approaches. In these *Delta State-Based Conflict-Free Replicated Data Types (δ -CRDTs)*, only relevant parts of the state are sent for synchronization steps, encoded in a delta. This prevents large message sizes compared to state-based CRDTs, while not needing the complex message delivery constraints of operation-based CRDTs due to delta merges being commutative, associative, and idempotent.

1.1 Research Gap

A P2P system with data objects replicated through δ -CRDTs uses gossip-like methods to propagate updates, and there exist different such dissemination strategies such as proactive (push), reactive (pull) and hybrid (push-pull) models, which likely introduce different trade-offs between message redundancy, internal synchronization overhead, and convergence speed.

Even though previous research has already established different models for delta dissemination [7, 8], a systematic comparison between gossip-based strategies in the context of δ -CRDTs, discussing their characteristics and impact, is missing in current research. We therefore aim to complement previous work on eventually consistent replication systems and improve the understanding of how different algorithms impact the efficiency of P2P systems that use δ -CRDTs as their means of replication.

To achieve this, we implemented a testbed application in which various experiments can be performed. Furthermore, analyzed the effect of different dissemination strategies on system performance. In detail, we will answer the following questions:

1. How much of an effect do various strategies have on network load, internal delta computation times, and convergence speed for δ -CRDT maintenance?
2. Which dissemination strategies should be chosen based on underlying overlay network topologies and application characteristics?

By answering these questions, we aim to provide actionable insights for system designers and other researchers, who are working on balancing the cost of maintaining consistency with responsiveness and scalability in their respective environments.

1.2 Related Work

We survey related research to this work in two main areas: literature on δ -CRDTs, and performance studies of gossiping protocols for message dissemination. These build the foundation for understanding the performance of replication algorithms in P2P settings.

1.2.1 Delta state-based CRDTs

Almeida et al. [6, 9] introduced the concept of δ -CRDTs as an optimization of state-based CRDTs, by defining delta-mutators, which return a delta-state to be merged locally and remotely. They propose an anti-entropy algorithm using pair-wise communication and acknowledgement buffers, while pushing deltas periodically to random peers. There is an argument to be made that this only works well for continuous and static synchronization patterns [7], and is therefore not really suitable for a P2P setting. They also do not discuss different ways of dissemination.

The work of Enes et al. [10] builds on top of this, improving the proposed algorithm by avoiding back-propagation and redundant state transmissions. They then proceed in executing experiments with different CRDT types and various network topologies, finding that the new algorithm improves bandwidth and CPU usage. The limitations regarding communication patterns and dissemination strategies, however, remain.

Van der Linde et al. [7] designed a different algorithm to compute and merge the delta encodings based on version vectors [11]. This was shown to require less data to be sent compared to traditional approaches, and more suitable for P2P settings. This approach was also included in the Legion framework [12] for client-side replication in web applications. Even though the algorithm includes push and pull-based models for dissemination, there is no study of how they compare. Nevertheless, this paper is fundamental for our research.

In 2020, Sanjuan et al. [8] explored a transport and persistence layer on top of CRDTs based on a *Directed Acyclic Graph* (DAG) called Merkle-DAG, making it easier for nodes to pull the missing history from other compute nodes. Their proposed general anti-entropy algorithm for MerkleCRDTs uses a push-based broadcast mechanism to make other nodes discover new updates and synchronize accordingly. This also works for δ -CRDTs, but there is also no discussion about different gossiping mechanisms for convergence.

1.2.2 Comparative Studies of Gossiping Protocols

Randomized gossip protocols have been studied extensively in previous research as a way to disseminate information. The work on randomized rumor spreading by Karp et al. [13] shows that a simple push-pull mechanism converges in $\mathcal{O}(\log n)$ rounds using $\mathcal{O}(n \log \log n)$ messages. The insights into this complexity are valuable guidance to this research, and motivate the exploration of hybrid dissemination strategies. For δ -CRDTs, however, the sizes of messages and the computational costs of smaller and bigger updates might introduce different trade-offs beyond those considered in this work.

Alberto Montresor [14] then further studied gossiping and epidemic protocols in 2017 and surveyed how different protocols can be tailored. The aforementioned limitations, however, are the same, as δ -CRDT dissemination has new considerations regarding other resource consumptions.

2 Method

To examine the impact of dissemination strategies on an eventually consistent replication system with δ -CRDTs, we conducted a set of quantitative experiments on a self-made testbed. The numerical nature of the metrics we chose to track in line with our research questions (see Section 2.4) makes such a controlled quantitative approach well suited for understanding effects on system performance. We opted to develop our own testbed to have control over the execution of the dissemination strategies and precise measurement of relevant metrics, something that existing open-source alternatives did not provide [12, 15]. We ran the experiments locally on an Acer Aspire A515-57 laptop, which has 16 GB of memory, a 12th Gen Intel®

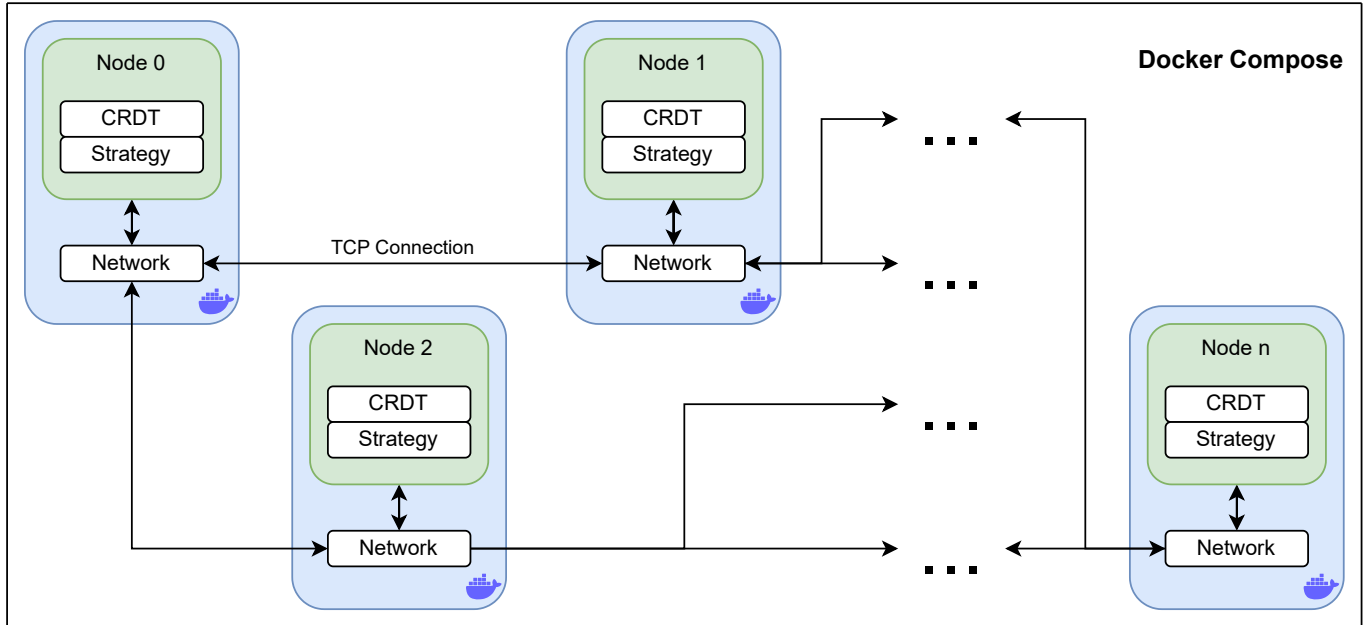


Figure 1: Architecture of the developed testbed, as implemented for the experiments. The connections between the nodes are dependent on the chosen overlay network.

Core™ i7-1255U processor, allowing for 12 threads to run in parallel. It runs Debian GNU/Linux 12 (Bookworm) operating system with kernel version 6.1.0-39-amd64. This choice was made due to the lack of alternative hardware resources during our research. In this section, we give further insight into the testbed architecture (Section 2.1), as well as overlay networks (Section 2.2), dissemination strategies (Section 2.3), and metrics (Section 2.4) for our experiments.

2.1 Testbed Architecture

We opted for a containerized architecture, with every peer running in its own Docker container [16], in our case managed by Docker Compose [17]. This allows for isolation, easy reproducibility of experiments, and better extensibility for future work. The testbed itself was written in Rust for its efficiency and reliable measurement of computational overhead.

Each node has a separate network layer, initiating and managing Transmission Control Protocol (TCP) channels for message exchange with its neighbors. The connections are based on an overlay network, which can be exchanged for different experiments (see Section 2.2). As the base of replication, an implementation of a δ -CRDT is part of all the nodes, and we can support various δ -CRDT implementations. For merging and retrieving the correct deltas, the peers manage a version vector, and the strategies determine when states are sent or exchanged. This modular approach allows tracking of network traffic, computation overhead, and convergence for different CRDT types without changing the actual replication mechanisms. Figure 1 shows an overview of the described architecture.

For the initial version of our testbed, we chose to implement an Observed-Remove Set (OR-Set). This is a textbook example of a CRDT replication system, so many researchers are already familiar with its structure and operations. It is also used by Van der Linde et al. [7] to showcase the efficiency of δ -CRDTs. It is a data structure that supports multiple operations (add and remove), while still being simple enough for the purpose of our experiment. Even though the type of CRDT has a large impact on message sizes and internal computation times for computing and merging deltas, we believe that this data structure is well suited to give an indication of dissemination strategy impact.

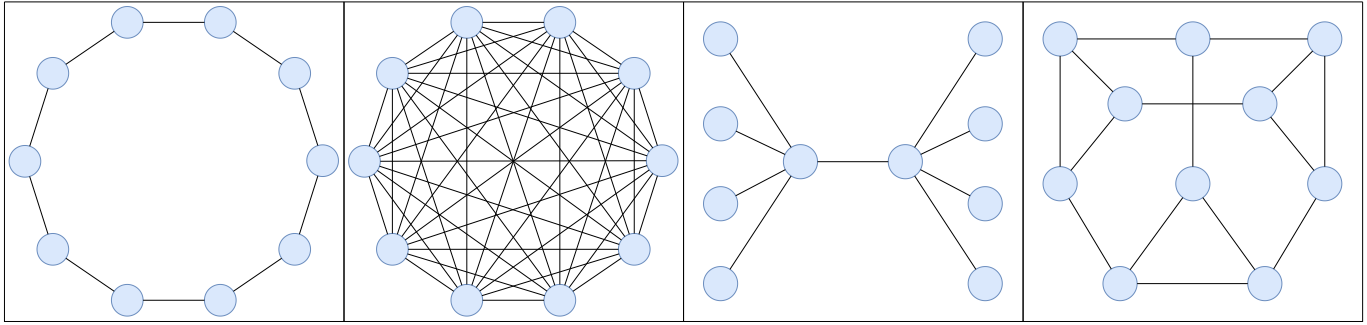


Figure 2: Set of overlay networks for the experiments.

2.2 Overlay Networks

For the evaluation of the dissemination strategies, we selected a representative set of overlay networks, as visible in Figure 2. To make sure that the process of every node can run in parallel on our equipment, we limited the number of nodes to ten for all structures.

1. **Ring Topology:** This is an example of a sparse overlay with a larger diameter, forcing updates to traverse through many nodes for dissemination. Furthermore, there exists little redundancy, and nodes have a low out-degree.
2. **Fully-connected Topology:** This is the densest possible overlay, with updates possibly being disseminated within a single hop. Here, we have a high redundancy and high node degree.
3. **Super-nodes Topology:** Without redundant paths for propagation, most of the load falls onto the few well-connected nodes. The degree of the peers is distributed highly unequally, and the diameter is still somewhat low.
4. **Mesh-like Topology:** Every node has a moderate degree and multiple redundant paths to disseminate updates throughout the network within a few hops.

Together, these topologies cover a large spectrum of overlay networks from sparse to dense, from centralized to decentralized, and from low diameter to high diameter, which allows us to study how network characteristics impact the effectiveness of different dissemination strategies. Due to time limitations, we constrained our analysis to these four networks, but the testbed application uses a properties file to define the underlying network topology, allowing it to use arbitrary topologies for further experimentation.

2.3 Dissemination Strategies

We implemented three strategies for delta dissemination that differ in terms of when and how the updates are distributed.

Proactive Strategy. In this strategy, whenever a replica updates its state, it immediately notifies all of its peers about the update by broadcasting its version vector. The receiving peers request a delta by answering with their own version vector before the original sender computes the delta, sends it over the network, where it is merged into the receiver's state. This merging is also treated as a local update, initiating another broadcasting sequence from the receiver. This strategy is somewhat similar to the mechanic used in the anti-entropy algorithm by Sanjuan et al. [8].

Reactive Strategy. Here, each replica sends periodic requests to a random peer to synchronize their state, again using version vectors. If one of the communication partners detects a state discrepancy, they compute and return the relevant delta to be merged. This was described as the pull-model by Van der Linde et al. [7]. Furthermore, we can change the number of peers a replica asks for updates at each timeout (fan-out).

Hybrid Strategy. This is a combination of the previous two strategies, where a replica pushes local updates to a set of its peers, while also periodically polling from neighbors. Each exchange is again a two-way synchronization as in the reactive strategy, and the number of peers contacted per exchange is also variable (fan-out). In contrast to the proactive strategy, a delta-merge operation does not trigger another exchange. This way of dissemination resembles the push-pull model in the aforementioned Van der Linde et al. paper [7].

Again, the goal was to cover a broader range of strategies to benchmark. For the scope of this project, a proactive strategy similar to push-based gossip, a reactive strategy similar to pull-based gossip, and a hybrid strategy based on push-pull-based gossip seemed like an adequate choice to provide a comparative evaluation.

2.4 Metrics Collection

We are able to collect the following metrics. All of them are written to CSV files after an experiment, and collected from the containers onto the host. This allows detailed quantitative comparisons after execution.

1. Network load

For every message a replica receives, the testbed records its size in bytes, and logs it for analysis after the run. We also log whether a message is redundant if it is not processed based on the local and remote version vector. This metric reflects the used bandwidth on the network that the application needs to operate, as well as the efficiency of network use.

2. Internal computation time

We obtain this metric by measuring the system time spent on every delta retrieval and delta-merge operation. This metric reflects the hardware requirements as well as the computational overhead of the described strategies.

3. Convergence speed

At each update, which can be either a local state change or a remote delta-merge, the nodes log the state of their version vectors together with a system timestamp. As all the nodes run on the same machine, we can use the system timestamps to reason about when a local update on one node is visible across all other replicas during post-processing.

These metrics line up with our described research questions, as the choice of strategy impacts the number of exchanged messages, the size of the deltas relevant for computation times, and the convergence speed, similar to existing research on gossiping, as outlined previously.

3 Results

We executed a number of experiments on our testbed application using different combinations of the following parameters:

- Network Topology (as described in 2.2)
- Dissemination Strategy (as described in 2.3)
- Fan-out (as described in 2.3)

We tested all the networks, all the strategies, and a total of three fan-outs for the reactive and hybrid strategy. Each node could either synchronize with only one random peer, all of its peers (fan-out = all), or a random logarithmic portion of its peers (fan-out = log). The number of peers in the last case is computed as:

$$N_{sync} = \lceil \log_2 N_{peers} \rceil$$

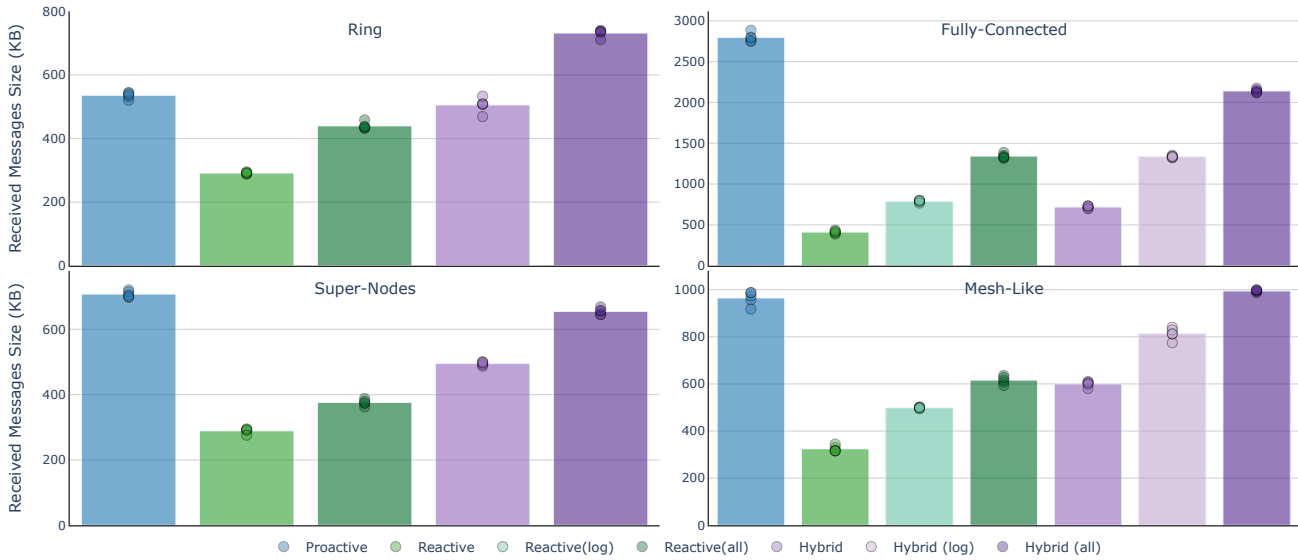


Figure 3: Total size of messages across strategies and topologies. Points show individual runs, and bars represent the mean of five runs.

Each node made a total of 30 updates throughout one experiment, using a jittered interval with a sample taken uniformly between 0 and 200ms. The same jitter was used for the synchronization interval in the reactive and hybrid strategies. After all the updates were completed, all nodes waited an additional three seconds, while handling further messages and initiating synchronization requests. All experiments were run five times.

In the following sections, we show the results for each of our metrics: network load (Section 3.1), computation times (Section 3.2), and convergence speed (Section 3.3).

3.1 Network Load

Figure 3 presents the total number of bytes transmitted over the network during the five runs of each experiment, along with the corresponding average. The reactive strategy with low fan-out requires the least bandwidth across all experiments, sending approximately 300 to 400 KB for every run. Particularly, for the fully-connected topology, the simple hybrid strategy also produced relatively low network load, only using 50-75% more bandwidth than the simple reactive strategy. Increasing the fan-out generally increased the total bytes sent, and a proactive strategy introduces high network load, especially for the fully-connected topology, inducing around seven times more network load than the reactive strategy.

Table 1 also presents the percentage of bytes present in messages that were not needed to perform state

Table 1: Average percentage of bytes in unnecessary messages per strategy and topology.

Strategy	Ring	Fully-Connected	Super-Nodes	Mesh-Like
Proactive	12.04%	50.19%	18.08%	24.14%
Reactive	26.04%	17.31%	24.88%	22.69%
Reactive (log)	–	31.24%	–	29.00%
Reactive (all)	31.59%	41.43%	30.21%	32.51%
Hybrid	41.03%	37.59%	41.64%	40.06%
Hybrid (log)	–	44.31%	–	42.86%
Hybrid (all)	43.48%	47.14%	42.64%	44.01%

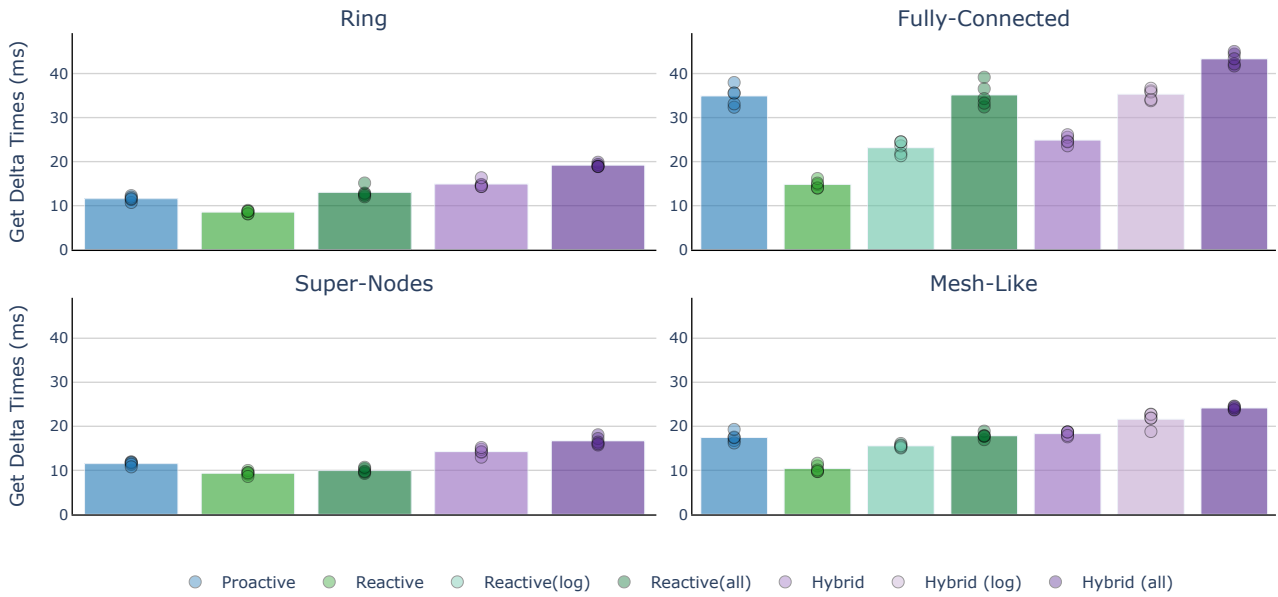


Figure 4: Aggregated time for retrieving deltas, across strategy and topology. Format as in Figure 3.

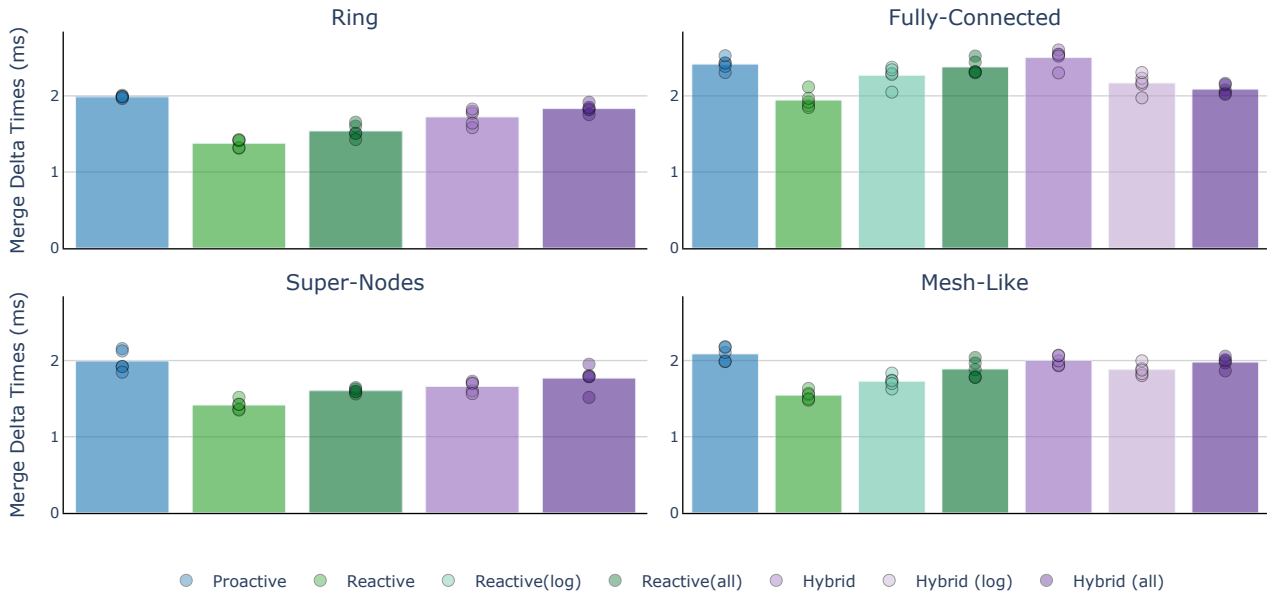


Figure 5: Aggregated time for merging deltas, across strategy and topology. Format as in Figure 3.

updates from the total amount of bytes sent over the network. It is noticeable that the hybrid strategies generally have a high redundancy of around 40% in this metric, getting worse for higher fan-out, whereas the proactive strategy has a very high redundancy in the fully connected topology (over 50%), but low redundancy elsewhere (under 25%). The simple reactive strategy has the lowest redundancy for the fully-connected and mesh-like topologies.

3.2 Computation Times

Figure 4 presents the aggregated time spent doing operations to retrieve requested deltas for all runs in combination with an average. There is not too much difference between the strategies for the ring and super-nodes topologies, but for the fully-connected and mesh-like topologies, the pull-based strategy with a fan-out of one spends the least time computing deltas (10-15ms). Increasing the fan-out to log and all again increases retrieval times, with the hybrid strategy using complete fan-out taking the largest computation time throughout all runs with up to 40ms in the fully-connected network.

Additionally, Figure 5 visualizes the aggregated time for delta merging across all runs. There is very

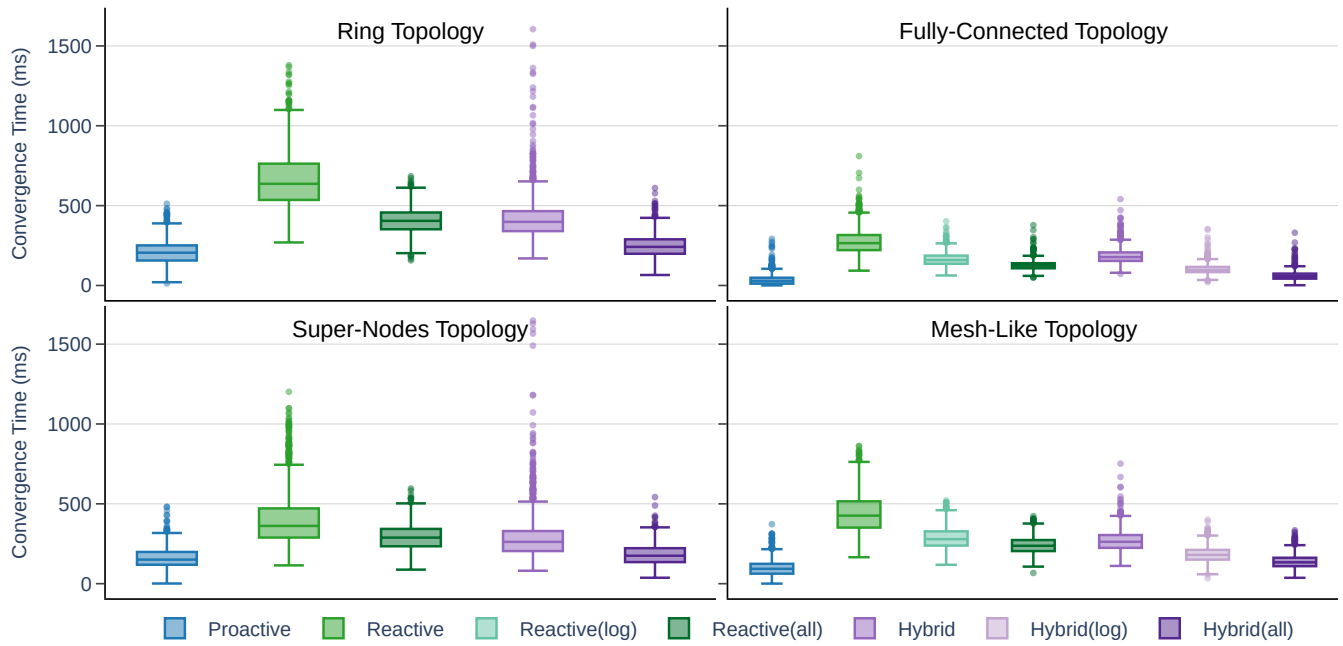


Figure 6: Distribution of dissemination times (single update) across strategies and topologies.

little difference between all networks and strategies, with the proactive strategy spending slightly more time, and the reactive strategy with fan-out of one slightly less time than the other strategies. In general, the internal computation times are ever so slightly skewed towards the fully-connected and mesh-like topologies.

3.3 Convergence Speed

Combined over all the runs, Figure 6 shows the distribution of the difference between the time of update initiation, and the time of full update dissemination. Across all topologies, the proactive strategy has the lowest median convergence time and a small variance, with no update taking longer than 500ms to be visible across all nodes. The reactive and hybrid strategies have higher median values, higher spreads, and more outliers if the fan-out is one, especially for the ring and super-node topologies where some updates take over 1.5 seconds to be completely disseminated. The convergence speed becomes faster and less variable, however, when increasing the fan-out to log, and this trend continues when setting the fan-out to all. Overall, the difference between the strategies was most visible for the ring topology and least visible for the fully-connected topology.

4 Discussion

In this section, we will reason about the results presented previously, and discuss limitations of our research. Overall, we can say that the choice of strategy has a big impact on network load, delta retrieval times, and convergence speed, which also depend on the network topology to varying degrees. Both topology and dissemination strategy have, however, little impact on delta merging times, which is due to the fact that always the same number of updates have to be merged. Having fewer, larger deltas is only marginally faster than many smaller ones for this CRDT type.

Proactive Strategy. Using this strategy with our various network topologies provided us with some interesting insights. Firstly, the proactive strategy has a high effect on network load, especially on dense networks, such as the fully connected or the mesh-like network. This occurs because the proactive strategy relies on broadcasting each update to ensure convergence. In contrast, in sparse networks, the proactive strategy performs more efficiently, showing lower network load and reduced message redundancy. With regard to computation times, the proactive strategy seems to result in higher delta computation times,

especially in dense networks. This is happening because each update triggers a delta calculation, which, depending on the nature of the calculation function, may be costly. We saw that generally, calculating fewer deltas for more updates per delta is more efficient than calculating a delta for each update. For convergence times, we see that the proactive strategy is dominant across all network topologies, particularly on sparse, high diameter networks.

Reactive Strategies. For the base case, where fan-out is one, the network load is consistently low and nearly independent of the used topology, while dominating the other strategies in this metric due to its timeout-based approach. Higher fan-outs seem to increase the network load almost linearly, by adding a lot of unnecessary messages. The regular reactive strategy also requires the least amount of time computing deltas, with an increased fan-out only having a negative effect for very dense networks, where deltas likely have more redundancies to handle. In terms of convergence speed, this method is the slowest throughout topologies. This is especially visible in high diameter networks, as every propagation hop takes at least one timeout. Increasing fan-out significantly improves median convergence speed and reduces variance, in particular for high diameter networks.

Hybrid Strategies. We again observe that without using any special fan-out, the network load is almost independent of the topology, which makes it comparably low for the denser networks with lower diameter. Higher fan-out greatly increases this, especially for these network types. A big portion of messages are unnecessary, however, independent of fan-out, and there is almost no fluctuation across topologies. Higher fan-out also leads to larger compute times for delta retrievals for dense networks, while not helping too much to bring down the convergence time, which is already quite low in the base case for more connected networks. For sparser, higher diameter and possibly imbalanced networks, the default hybrid strategy suffers a lot from a high variance.

Design Choices. Depending on the different characteristics of the application at hand, different dissemination strategies may be preferable. The proactive strategy performs best in sparse networks, where its typically high network usage remains manageable. It is very good for cases where the calculation of deltas is not very computationally heavy or does not scale well anyway. It is also the strategy to use when convergence time is the primary objective. The reactive strategy is good for applications with large, dense networks, as it is the most effective in network use. Tuning the fan-out can also have a big positive impact on the convergence time. Hybrid strategies are more suited for applications that require a balance between fast convergence and controlled network overhead. By dynamically tuning the proactive and reactive part of the communication, such strategies can adapt to varying network conditions or workload characteristics.

Limitations. Due to time and hardware constraints, we could not consider large topologies, distributed over multiple machines, which would have also included interesting variables for network latency, message loss, and possible network partitions. This makes it unclear, how accurately the results of the network loads will reflect the performance of real systems. Our approach for computing convergence times also would not work for multiple machines due to possible clock drift. There are likely some optimizations for our implementation of the OR-Set δ -CRDT for faster delta handling. Particularly using Merkle CRDTs [8] should improve delta computation times especially for smaller deltas, but implementation would not have been feasible within our time frame. We could also have considered more diverse strategies with a better methodology in a larger project, as well as a better notion of redundancy within deltas and different parameter settings for update creation and synchronization timeouts.

5 Conclusion and Outlook

Our experiments demonstrated the significant effect of varying dissemination strategies on efficiency given different overlay networks for δ -CRDT replication in a P2P system. The proactive strategy we implemented achieved the fastest convergence throughout, but had high network and computation overhead in dense topologies. Reactive strategies minimized these factors at the cost of slower synchronization, which can be mitigated somewhat by adding larger fan-outs. Hybrid strategies offer another balance between the two approaches, but also suffer from high message redundancy.

For future work, we invite researchers to extend our experiments to larger, distributed networks, better reflecting real-world systems. Other topics to investigate could be adaptive strategies that change based on the current system load, and more complex types of δ -CRDTs with different characteristics for delta retrieval and delta-merge operations.

References

- [1] E. Brewer, “Cap twelve years later: How the ”rules” have changed,” *Computer*, vol. 45, no. 2, pp. 23–29, 2012. doi: 10.1109/MC.2012.37. [Online]. Available: <https://doi.org/10.1109/MC.2012.37>
- [2] W. Vogels, “Eventually consistent,” *Commun. ACM*, vol. 52, no. 1, p. 40–44, Jan. 2009. doi: 10.1145/1435417.1435432. [Online]. Available: <https://doi.org/10.1145/1435417.1435432>
- [3] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. Philadelphia, PA: USENIX Association, Jun. 2014. ISBN 978-1-931971-10-2 pp. 305–319. [Online]. Available: <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [4] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Stabilization, Safety, and Security of Distributed Systems*, X. Défago, F. Petit, and V. Villain, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011. ISBN 978-3-642-24550-3 pp. 386–400. [Online]. Available: https://doi.org/10.1007/978-3-642-24550-3_29
- [5] N. Preguiça, “Conflict-free replicated data types: An overview,” 2018. [Online]. Available: <https://doi.org/10.48550/arXiv.1806.10254>
- [6] P. S. Almeida, A. Shoker, and C. Baquero, “Delta state replicated data types,” *Journal of Parallel and Distributed Computing*, vol. 111, pp. 162–173, 2018. doi: 10.1016/j.jpdc.2017.08.003. [Online]. Available: <https://doi.org/10.1016/j.jpdc.2017.08.003>
- [7] A. van der Linde, J. a. Leitão, and N. Preguiça, “ δ -crdts: making δ -crdts delta-based,” in *Proceedings of the 2nd Workshop on the Principles and Practice of Consistency for Distributed Data*, ser. PaPoC ’16. New York, NY, USA: Association for Computing Machinery, 2016. doi: 10.1145/2911151.2911163. ISBN 9781450342964. [Online]. Available: <https://doi.org/10.1145/2911151.2911163>
- [8] H. Sanjuan, S. Poyhtari, P. Teixeira, and I. Psaras, “Merkle-crdts: Merkle-dags meet crdts,” 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2004.00107>
- [9] P. S. Almeida, A. Shoker, and C. Baquero, “Efficient state-based crdts by delta-mutation,” in *Networked Systems*, A. Bouajjani and H. Fauconnier, Eds. Cham: Springer International Publishing, 2015. ISBN 978-3-319-26850-7 pp. 62–76. [Online]. Available: https://doi.org/10.1007/978-3-319-26850-7_5
- [10] V. Enes, P. S. Almeida, C. Baquero, and J. Leitão, “Efficient synchronization of state-based crdts,” in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, 2019. doi: 10.1109/ICDE.2019.00022 pp. 148–159. [Online]. Available: <https://doi.org/10.1109/ICDE.2019.00022>
- [11] D. Parker, G. Popek, G. Rudisin, A. Stoughton, B. Walker, E. Walton, J. Chow, D. Edwards, S. Kiser, and C. Kline, “Detection of mutual inconsistency in distributed systems,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 3, pp. 240–247, 1983. doi: 10.1109/TSE.1983.236733. [Online]. Available: <https://doi.org/10.1109/TSE.1983.236733>

- [12] A. van der Linde, P. Fouto, J. a. Leitão, N. Preguiça, S. Castiñeira, and A. Bieniusa, “Legion: Enriching internet services with peer-to-peer interactions,” in *Proceedings of the 26th International Conference on World Wide Web*, ser. WWW ’17. Republic and Canton of Geneva, CHE: International World Wide Web Conferences Steering Committee, 2017. doi: 10.1145/3038912.3052673. ISBN 9781450349130 p. 283–292. [Online]. Available: <https://doi.org/10.1145/3038912.3052673>
- [13] R. Karp, C. Schindelhauer, S. Shenker, and B. Vocking, “Randomized rumor spreading,” in *Proceedings 41st Annual Symposium on Foundations of Computer Science*, 2000. doi: 10.1109/SFCS.2000.892324 pp. 565–574. [Online]. Available: <https://doi.org/10.1109/SFCS.2000.892324>
- [14] A. Montresor, *Gossip and Epidemic Protocols*. John Wiley & Sons, Ltd, 2017, pp. 1–15. ISBN 9780471346081. [Online]. Available: <https://doi.org/10.1002/047134608X.W8353>
- [15] Y. Zhu and J. Ma, “Crust: A modular framework for conflict-free replicated data types (crdts) development, validation, and benchmarking,” *IEEE Access*, vol. 13, pp. 84 081–84 093, 2025. doi: 10.1109/ACCESS.2025.3568925. [Online]. Available: <https://doi.org/10.1109/ACCESS.2025.3568925>
- [16] “Docker engine,” accessed: 2025-09-26. [Online]. Available: <https://docs.docker.com/engine/>
- [17] “Docker compose,” accessed: 2025-09-26. [Online]. Available: <https://docs.docker.com/compose/>

A List of Acronyms and Abbreviations

δ -CRDT Delta State-Based Conflict-Free Replicated Data Type

CRDT Conflict-Free Replicated Data Type

DAG Directed Acyclic Graph

OR-Set Observed-Remove Set

P2P Peer-to-Peer

TCP Transmission Control Protocol